

● *VidFlow*

Technical Documentation & Glossary

Document 2 of 2. The original project proposal, why each piece of technology was chosen over its alternatives, how it all fits together in plain English, and a glossary of the technical terms used throughout the build.

Prepared for	Okezie Collington
Prepared by	Claude (Anthropic), Claude Code session
Date	16 September 2026
Repository	C:\Users\USER\Desktop\VidFlow App
Companion document	"VidFlow — Pending Tasks & Handoff Status" (Document 1 of 2)

Contents

1. Original Project Proposal

2. Technology Stack & Why Each Choice Was Made

3. How It All Fits Together (Plain English)

4. Glossary of Technical Terms

1. Original Project Proposal

This is the plan VidFlow was actually built against — the brief given at the start of the project, restated here so it travels alongside the finished code.

What VidFlow is: a single platform combining four things that are normally separate products — screen/video recording (like Loom), live meetings and webinars (like Zoom), a recruitment pipeline with video interviews (like a mini applicant-tracking system), and a training/course platform (like a lightweight LMS) — plus an AI assistant layered across all of them.

Who it's for: teams and businesses who currently juggle 3-4 separate subscriptions to cover these needs, and would rather have one account, one video library, and one AI layer over everything they record.

The build plan

The specification called for the platform to be delivered in stages, each one a complete, working slice rather than one large all-at-once build — so the product is always in a demonstrable state:

- **Foundation** — accounts, permission levels (Owner/Admin/Member), and Free vs. Pro plans.
- **Recording** — a browser-based screen and camera recorder, then an advanced version with drawing tools, a webcam "bubble," and privacy blur.
- **Recruitment** — job postings, video interview questions, and a pipeline for reviewing candidates.
- **Training** — courses built from modules and lessons, with progress tracking and quizzes.
- **Live** — real-time meetings and large-audience webinars.
- **AI** — automatic titles, summaries, transcripts, and quizzes generated from any video.
- **Reach** — a mobile app and a Chrome browser extension, so recording isn't limited to visiting the website.

Every stage was to be built with real, working code — not mockups or instructions to implement later — verified with automated tests, and documented as it was built, so the project could be picked up by anyone (including another developer or another AI coding tool) at any point without needing to reverse-engineer decisions from scratch.

Status against this proposal: every stage listed above has been built and verified. What remains is entirely account setup and deployment — covered in the companion document, "*VidFlow — Pending Tasks & Handoff Status.*"

2. Technology Stack & Why Each Choice Was Made

Every major building block below was chosen deliberately, usually against one or two well-known alternatives. Unfamiliar terms are explained in plain English here and defined again in the Glossary (Section 4).

Next.js PICKED

The framework the whole website is built on — it handles pages, navigation, and talking to the server all in one system, using React (a very widely used way of building web interfaces).

Compared against: a plain React app + separate backend (e.g. Express), or older frameworks like Create React App. Next.js was preferred because it bundles the website and the server-side logic into one project, which is simpler to build, deploy, and reason about for a solo/small build — no need to run and coordinate two separate applications.

Supabase PICKED

Provides the database (where all information — users, videos, job applications, courses — is permanently stored), user login/sign-up, and fine-grained data permissions, as one connected service.

Compared against: Firebase (Google's equivalent), or assembling a database + auth system manually. Supabase was preferred because it uses PostgreSQL — a standard, open, and extremely well-understood database technology — rather than a proprietary format, and its "Row Level Security" feature lets permission rules live directly in the database itself, rather than being an afterthought bolted onto the code.

Cloudflare R2 PICKED

Where the actual video files are stored (the database stores information *about* videos, not the large video files themselves).

Compared against: Amazon S3 (the industry standard). R2 was preferred specifically because it charges nothing to *send data out* (what's called "egress"), which is normally one of the largest ongoing costs for a video product — every time someone watches a video, that's an egress transfer. R2 also speaks the exact same technical language as S3, so switching later, if ever needed, is straightforward.

LiveKit PICKED

Powers real-time live meetings and webinars — the technology that lets multiple people see and hear each other with minimal delay.

Compared against: Zoom's SDK, Twilio, or Daily.co. LiveKit was preferred because it's open-source (inspectable, and can be self-hosted later at no licensing cost if the project ever needs to leave the cloud version), and it was built specifically to scale from small meetings up to large one-to-many webinars using the same underlying technology.

Google Gemini PICKED

The AI model behind the AI Assistant feature — generating titles, summaries, transcripts, and quizzes from a recorded video.

Compared against: OpenAI's Whisper + GPT combination (a separate transcription step, then a separate summarization step). Gemini was preferred because it can watch and listen to a video file directly and understand both the visuals and the audio together in a single step, rather than needing the video converted to text first and then processed separately — fewer moving parts, and it can reference on-screen content, not just spoken words.

Tailwind CSS + shadcn/ui PICKED

The system used to style every button, page, and component so the whole product looks consistent.

Compared against: hand-written CSS files, or a pre-built component library like Material UI. Tailwind was preferred because styles live directly alongside the code that uses them (nothing to hunt for in a separate file), and shadcn/ui provides ready-made components (buttons, dialogs, forms) as code that lives inside this project — fully customizable — rather than as a locked black-box package.

TypeScript PICKED

A stricter version of JavaScript (the language the whole product is written in) that catches a large class of mistakes before the code ever runs.

Compared against: plain JavaScript. TypeScript was preferred because it flags an entire category of bugs — like accidentally treating a number as text, or misspelling a field name — at the moment code is written, rather than waiting for a user to trigger the bug in production.

Capacitor PICKED

Wraps the existing website into installable Android and iPhone apps, rather than building the mobile apps as separate projects from scratch.

Compared against: React Native, or fully native Swift/Kotlin apps. Capacitor was preferred because it reuses the already-built website almost entirely — one codebase serves the website, Android app, and iPhone app — which is dramatically less work to build and keep updated than maintaining three separate versions of the product.

No ORM / Prisma DELIBERATELY NOT USED

An ORM ("Object-Relational Mapper" — Prisma is the most popular example) is a tool that lets a developer write database queries using the programming language directly instead of writing raw database commands.

Why it was skipped here: Supabase already generates a fully-typed, safe way to query the database directly from the code (so TypeScript still catches mistakes), and it works hand-in-hand with Supabase's Row Level Security permission system. Adding Prisma on top would mean maintaining two separate descriptions of the database schema that have to be kept in sync — extra complexity with no real benefit for this project.

3. How It All Fits Together (Plain English)

No jargon — just what happens, in order, when someone uses VidFlow.

Recording and watching a video

Someone clicks "Record" in their browser (or the Chrome extension). Their screen and/or camera is captured directly in the browser, split into small pieces as it records, and each piece is uploaded in the background — so a recording that runs long, or a shaky internet connection, doesn't lose the whole video. Once finished, the pieces are stitched into one file and stored in Cloudflare R2 (Section 2). A record of the video — its title, owner, and a link to where the file lives — is saved in the Supabase (Section 2) database. Anyone with the share link opens a page that streams the video back from storage.

Signing up and permissions

Supabase handles the actual login process (checking passwords, issuing secure sessions). Once logged in, what a person can see and do is controlled by two things working together: their *role* (Owner, Admin, or Member) and the database's own permission rules, which check — for every single piece of data — whether the person asking is actually allowed to see it. This means the permission check happens at the database itself, not just in the website's code, so there's no single place where a mistake could accidentally expose someone else's data.

A live meeting

When someone starts a meeting, the website asks LiveKit (Section 2) to set up a live "room." Everyone who joins connects to that room directly for audio and video, while the website itself just handles who's allowed in and the surrounding controls (chat, host permissions). This separation is why meetings can scale to a large webinar audience without slowing down the rest of the website.

The AI assistant

After a video finishes uploading, it can be sent to Google Gemini (Section 2), which watches and listens to it directly and returns a title, summary, transcript, or quiz — whichever was requested. That result is saved back into the database, next to the video, so it only needs to be generated once.

Mobile app and Chrome extension

The mobile app isn't a separate product — it's the same website, wrapped so it can be installed like an app and access phone features like the camera. The Chrome extension is a small, separate program that runs inside the browser toolbar, captures screen or camera recordings locally, and hands the finished video to VidFlow exactly the same way the website's own recorder does.

4. Glossary of Technical Terms

Plain-English definitions for the technical terms that came up throughout this build.

npm

Short for "Node Package Manager." Almost no modern web project is written entirely from scratch — npm is the tool used to download and manage all the small, reusable pieces of code (called "packages") that a project depends on, and to keep track of exactly which versions are in use.

Package / dependency

A pre-written piece of code, written by someone else, that this project uses instead of reinventing. For example, the code that talks to Cloudflare R2 is a package, not something written from scratch here.

Prisma / ORM

An ORM ("Object-Relational Mapper") is a tool that translates code into database commands automatically. Prisma is the best-known example. VidFlow deliberately does not use one — see Section 2 for why.

API

Short for "Application Programming Interface" — a defined way for two pieces of software to talk to each other. When VidFlow's website asks Google Gemini to summarize a video, that request travels through Gemini's API.

Database / PostgreSQL

The permanent storage system that holds all of VidFlow's information — every user, video record, job application, and course. PostgreSQL (often called "Postgres") is the specific, widely-used database technology that Supabase is built on top of.

Authentication ("auth")

The process of proving who someone is — typically by checking an email and password — before letting them into an account.

Row Level Security (RLS)

A feature of the database itself (not the website's code) that checks, for every single request, whether the person asking is actually allowed to see or change that specific piece of data.

Environment variable / `.env.local`

A setting — usually a secret key or password — that the code reads at startup, kept in a separate file rather than written directly into the code, so secrets are never accidentally shared or uploaded to GitHub.

Git / GitHub

Git is a tool that saves a complete history of every change made to the code, so nothing is ever truly lost and changes can be undone. GitHub is a website that stores that history online and lets other tools (like a hosting service) access it.

Repository ("repo")

The full folder of project code and its entire saved history, as tracked by Git.

Deployment / hosting (Netlify)

Deployment is the process of taking the code and putting a live, running copy of it on the internet where the public can visit it. Netlify is the specific hosting service chosen to do this for VidFlow.

SDK

Short for "Software Development Kit" — a ready-made package, provided by a company like Google or Amazon, that makes it easier to use their service from code without handling all the low-level technical details yourself.

Framework

A large, structured starting point for building software, which handles common needs (like moving between pages) so they don't need to be built from scratch. Next.js is VidFlow's framework.

Server / client

"Server" refers to code that runs on a remote computer (safe from prying eyes — this is where secret keys and database access happen). "Client" refers to code that runs in the visitor's own browser. VidFlow constantly moves work between the two, always keeping secrets on the server side.

SFU (Selective Forwarding Unit)

The technical approach LiveKit uses to relay live audio and video between meeting participants efficiently, rather than sending every person's stream directly to every other person.

Multipart upload

A method of uploading a large file (like a long recording) in several smaller pieces instead of one single transfer — so a lost connection only requires retrying the failed piece, not the whole file.

Manifest V3

The current technical ruleset Google requires all Chrome extensions to follow. VidFlow's Chrome extension is built to this standard.

Service worker

A small piece of background code (used by the Chrome extension) that can run even when no visible window is open, listening for and coordinating tasks like starting a recording.

Personal access token

A long, randomly generated code that acts like a password specifically for one tool (like the Chrome extension) to identify itself to VidFlow, without needing to type a real password into it.

Migration (database)

A saved, numbered script that makes one specific change to the database's structure (like adding a new table). Running all of VidFlow's migrations in order builds the complete database from nothing.

Linting / type-checking

Automated tools that read through the code looking for likely mistakes or inconsistent style, without actually running it — catching problems before they ever reach a real user.

Automated tests

Small programs that check whether a specific piece of the product still behaves correctly, run automatically every time the code changes, to catch anything broken before it ships.